LECTURE 4

DEEP REPRESENTATION LEARNING I

What is Learning?

PRESS → OR SPACE.

© Marc Rußwurm
Licensed under **Creative Commons Attribution–NonCommercial 4.0 International (CC BY-NC 4.0)**.
You may share and adapt these slides for teaching, research, and other non-commercial educational purposes with attribution.
Commercial training, paid workshops, consulting seminars, or incorporation into commercial course products require prior permission.

UNIVERSITÄT BONN

Loading slide...

Loading slide...

Today: Lecture Structure

Three major blocks guide the narrative.

1

Why Learn Representations?

2

Multilayer Perceptrons

3

Gradient Descent & Backpropagation

Start with representation quality and geometric separability.

Why Learn Representations?

Good representations make hard problems easy.

From manual feature design to learned representations.

A difficult task is easy in the right form.

Recap Lecture 1

CHINESE	□们的心智已经学会理解这个句子。
ARABIC	لقد تعلّم عقلنا أن يفهم هذه الجملة.
HEBREW	המוח שלנו למד להבין את המשפט הזה.
KOREAN	우리의 마음은 이 문장을 이해하는 법을 배웠습니다.
LATIN	Our mind has learned to understand this sentence.

Photo by Vitaly Gariev on Pexels

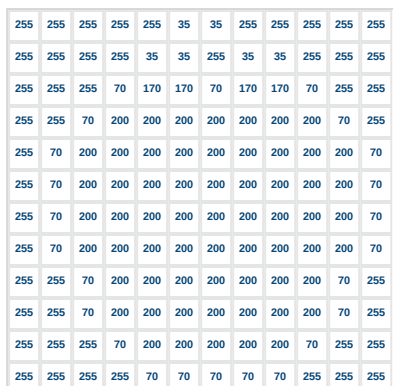
Why Learn Representations? Multilayer Perceptrons Gradient Descent & Backpropagation

UNIVERSITÄT BONN

Choosing Suitable Data Representations

Recap Lecture 1

Images



Math

```
XLVII + LXXVIII = ?
```

```
47 + 78 = 125
```

Places

COORDINATES

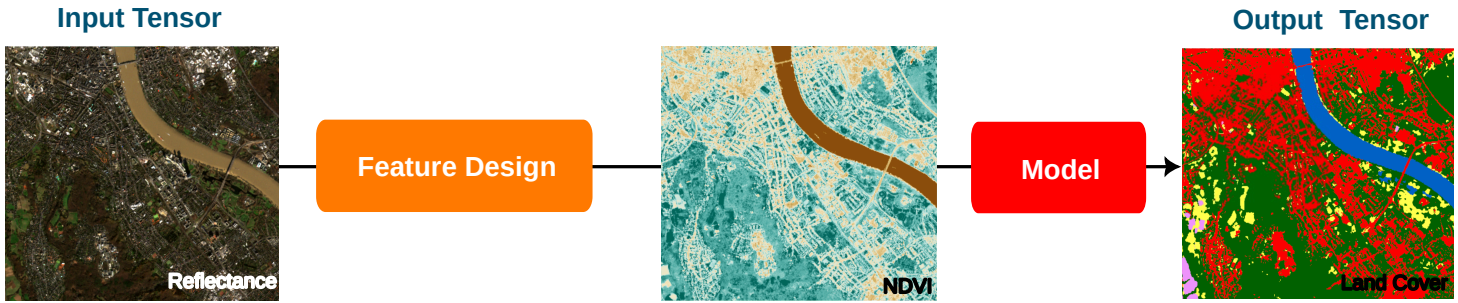
50°43'57.73" N,
7°06'16.63" E

PLACE

Hofgarten, Bonn, Germany

Machine Learning needs good Features

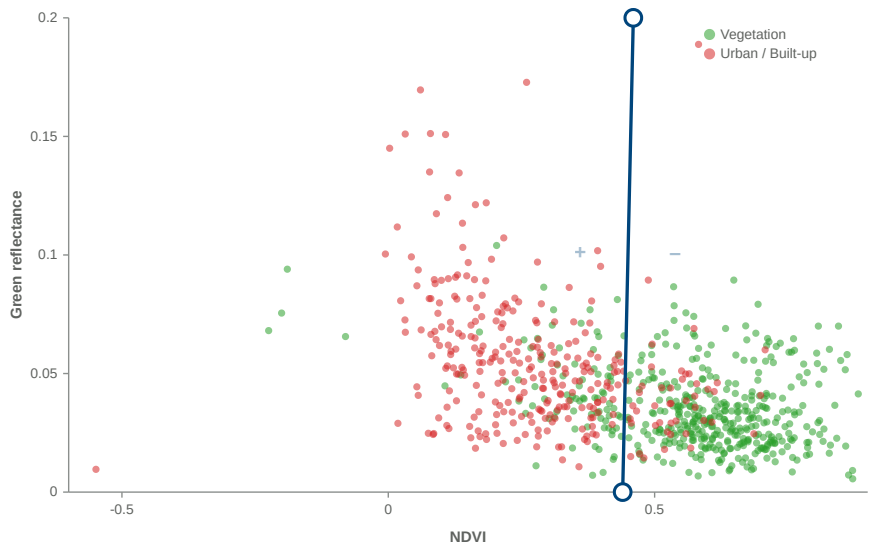
Recap Lecture 3



Raw Band: Green

Engineered Feature: NDVI

Good Representations allow for Simple Models



LINEAR REGRESSION MODEL

$$y = w_1\,\mathrm{NDVI} + w_2\,\mathrm{Green} + b$$

$$y = \mathbf{w}^\top \mathbf{x} + b$$

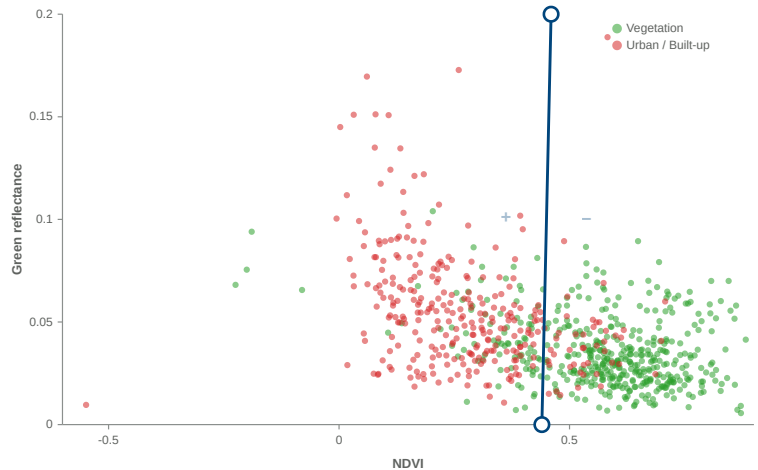
CURRENT BOUNDARY

$$y = -1.00\,\mathrm{NDVI} + 0.10\,\mathrm{Green} + 0.44$$

Accuracy **82.9%** (759 points)

Drag either handle to reposition the line. Hover the plot to measure distance to the boundary.

Logistic Regression for Classification



LOGISTIC REGRESSION MODEL $P(y \mid \mathbf{x})$

$$P(y{=}1 \mid \mathbf{x}) = \sigma(w_1\,\mathrm{NDVI} + w_2\,\mathrm{Green} + b)$$

$$P(y{=}1 \mid \mathbf{x}) = \sigma\big(\mathbf{w}^\top \mathbf{x} + b\big)$$

where $\sigma(z) = \dfrac{1}{1+e^{-z}}$ squashes the score into a probability in (0, 1).

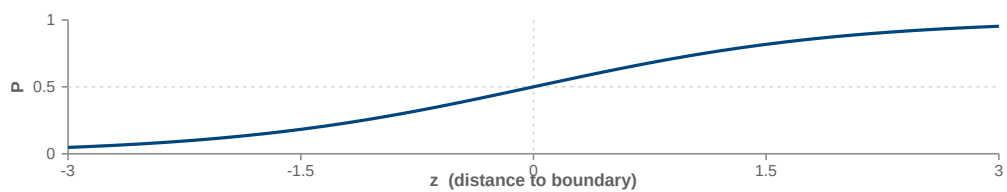
DECISION BOUNDARY · P = 0.5

$$z = -1.00\,\mathrm{NDVI} + 0.10\,\mathrm{Green} + 0.44$$

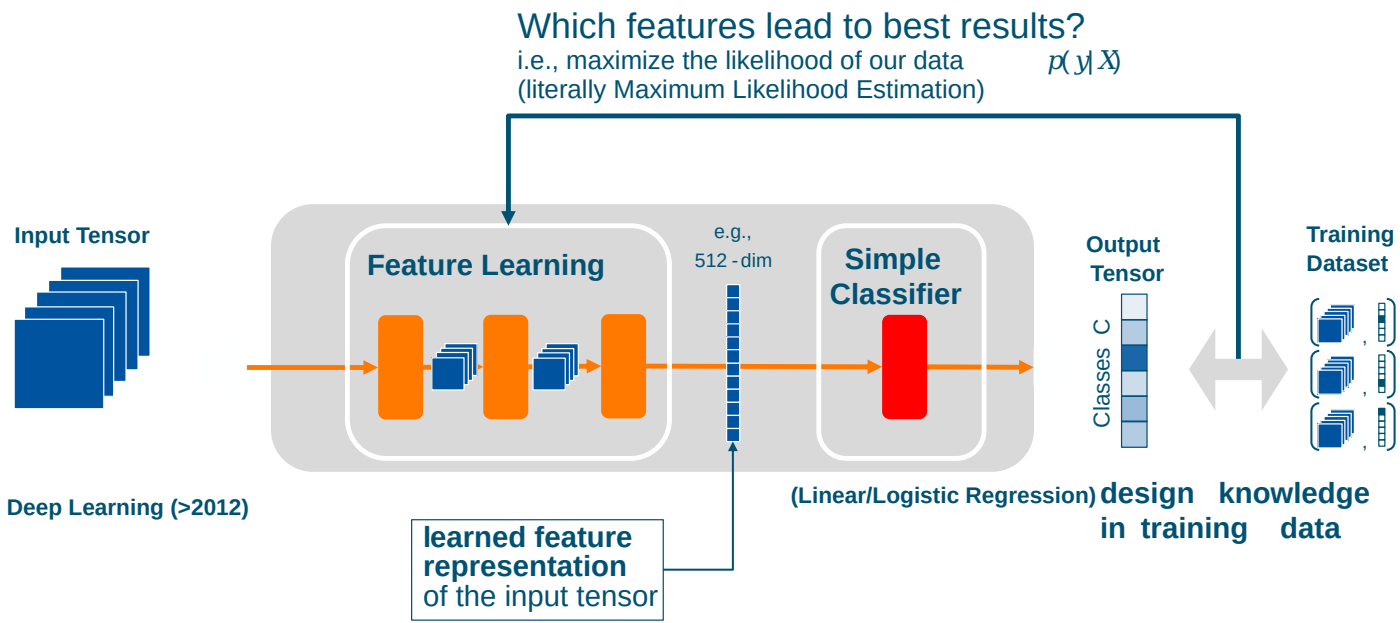
Accuracy **82.9%** (759 points)

Drag either handle to reposition the boundary. Hover the plot to read the predicted probability.

The logistic (sigmoid) function — squashing the score *z* into a probability



Learning Feature Representations

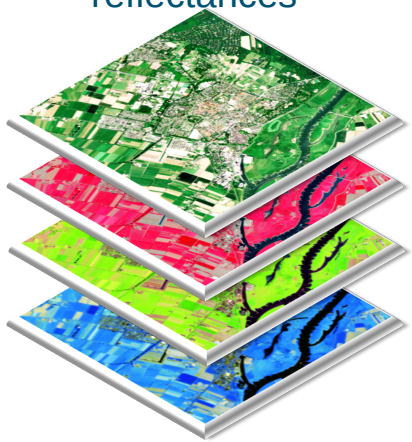


Timeline and Evolution of Deep Learning

<2012	2015 onwards	>2020 →
Classic Machine Learning	**Supervised Deep Learning**	**Self-Supervised Learning**
Feature Design: Which features extract the most information?	**Model Design:** Which architecture is best for my tensors?	**Loss Design:** Which loss function or training strategy?
Data: small labelled	**Data:** large labelled	**Data:** large unlabelled
Models: Random Forest, Boosting, Lin. Regression	**Models:** MLPs, CNNs, RNNs	**Model:** Transformers
Covered in Lectures 2 & 3	***Covered in Lectures 4 & 5***	***Covered from Lecture 7 onwards***

Image Classification

Input Tensor
reflectances



[H x W x D]

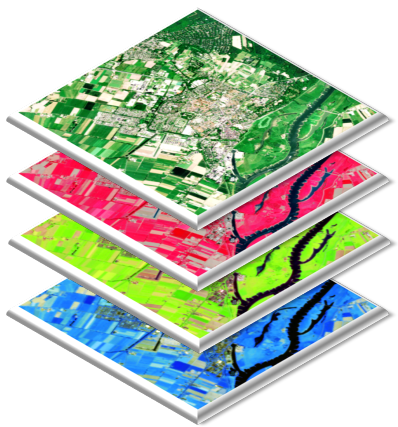
Image Classification

(Deep) Model

Output Tensor
class probabilities

[C]

Image Segmentation

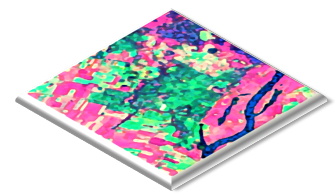


[H x W x D]

Image Segmentation

(Deep) Model

Output Tensor
segmentation map



[H x W x C]

Object Detection

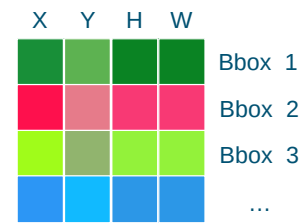
Input Tensor
reflectances

[H x W x 3]

Object Detection

(Deep) Model

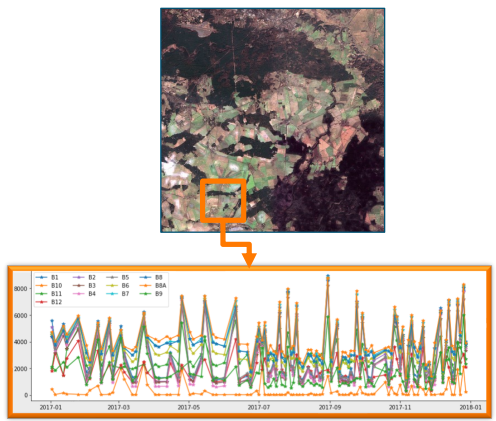
Output Tensor
bounding boxes



[N x 4]

Time Series Classification

[T x H x W x D]



[T x D]

Time Series Classification

(Deep) Model

e.g., classes

[C]

Overview: Deep Model Architectures

Lab 4 Content

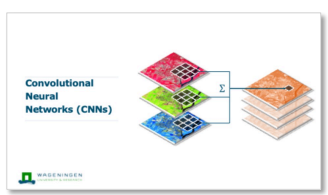
1950s | 1990s | 2015 | 2020s

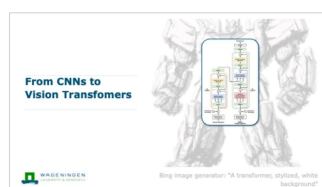
MLP
Linear Projections
Activation Functions
Linear Algebra &
Biological Analogy
Parallelism on GPUs

CNN
Convolutions
Pooling
Recep. Fields

ResNet
Deeper Networks
Batch Normalization
Skip Connections
Classification &
Segmentation Tasks

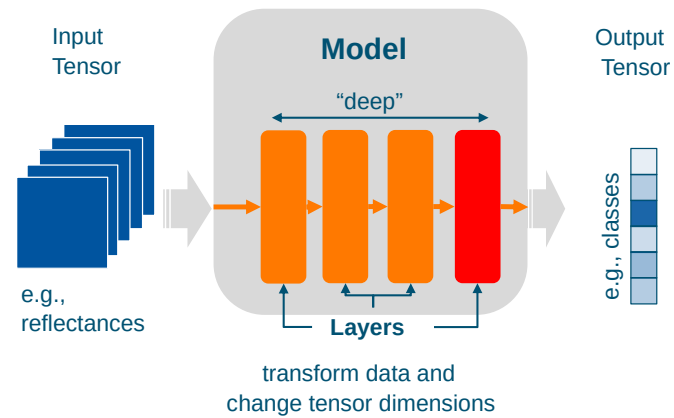
Transformers
Attention
Mechanism
Self-Attention (SA)
Multi-Head SA
Transformer Layers



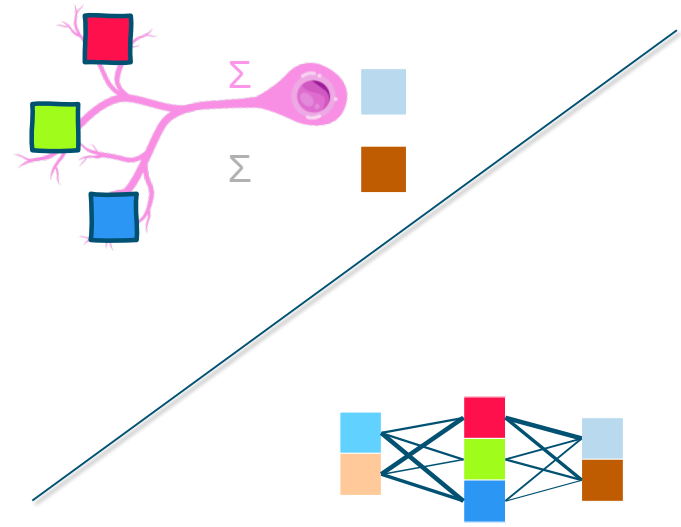


Recap: Learned Representations

- Good representations make difficult problems easier.
- In classic machine learning, we manually select data and engineer features to be a good representation of the problem.
- Deep learning models learn a feature representation that is then well-suited for a final linear classification with logistic regression.



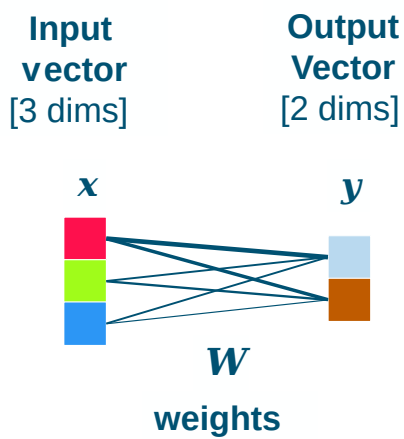
Multilayer Perceptrons

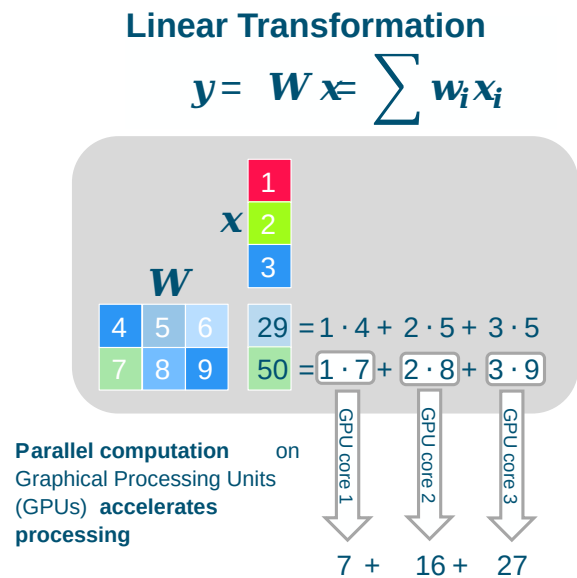


Learning the representation.

From linear decisions on fixed features to learned feature-space transformations.

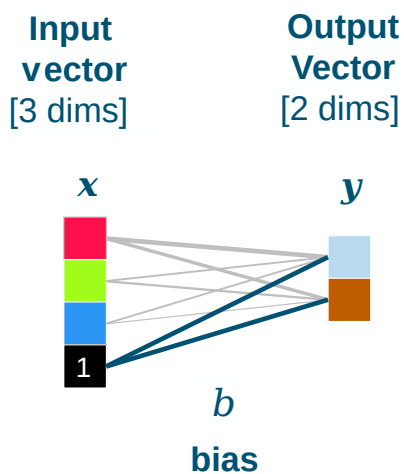
Linear Transformation

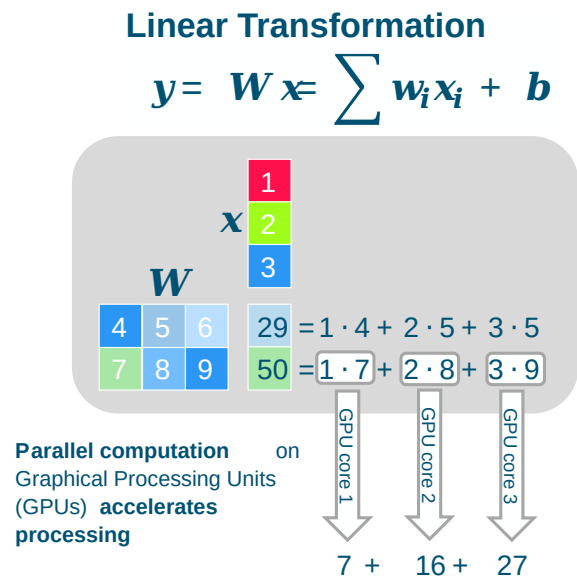




Parallel computation on Graphical Processing Units (GPUs) **accelerates processing**

Linear Transformation with Bias

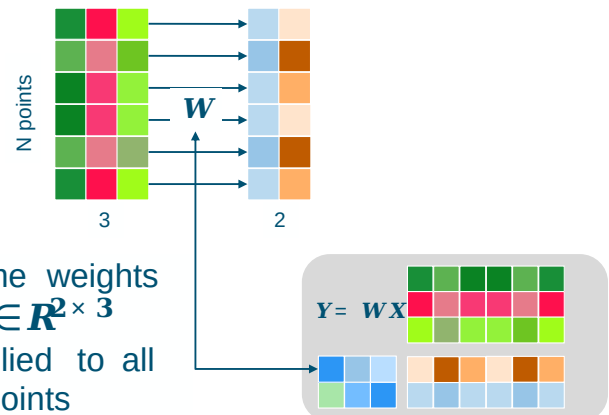




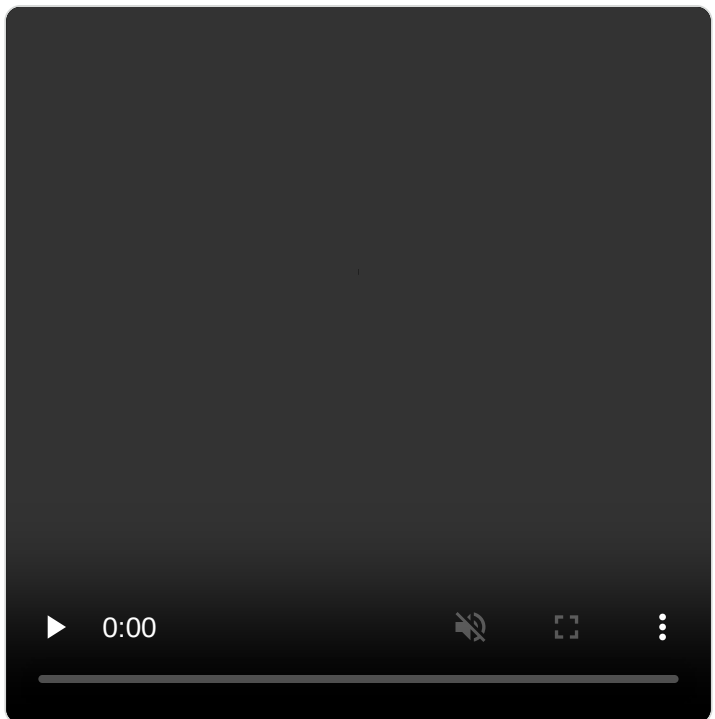
Parallel computation on Graphical Processing Units (GPUs) **accelerates processing**

Linear Transformation from 3D to 2D

Linear Transformation
of N points from 3D to 2D

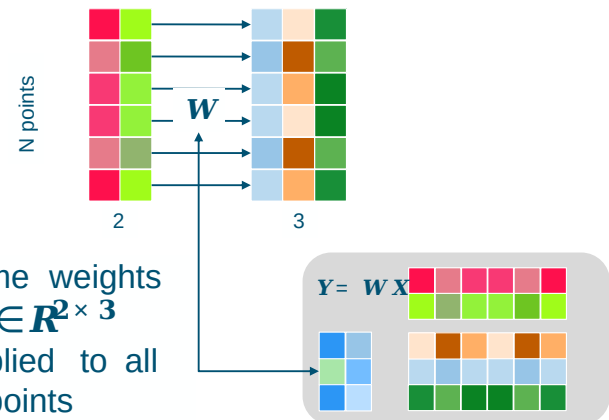


same weights
$W \in R^{2 \times 3}$
applied to all
N points

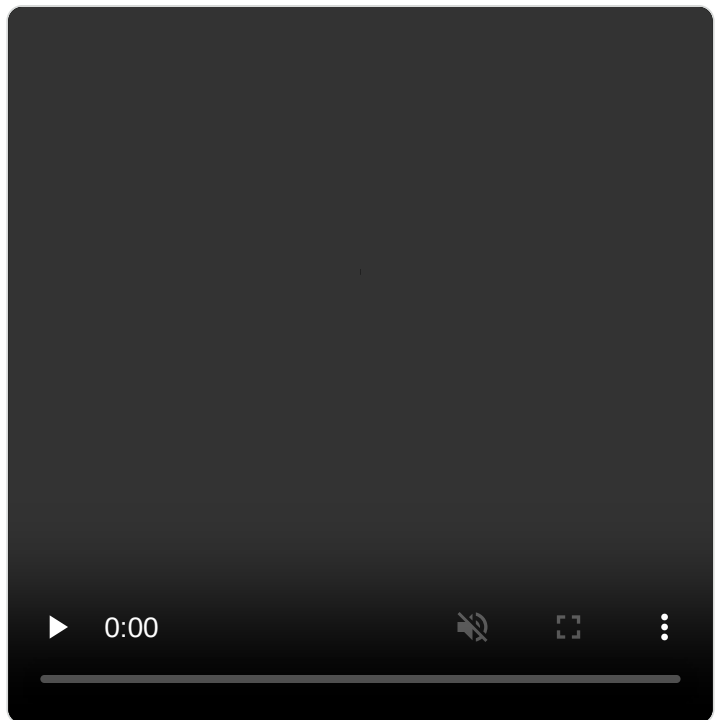


Linear Transformation from 2D to 3D

Linear Transformation $Y = WX$
of N points from 2D to 3D

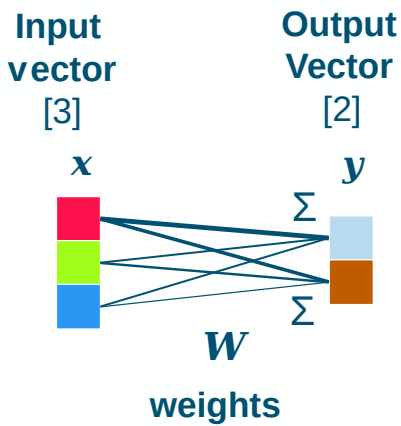


same weights
$W \in R^{2 \times 3}$
applied to all
N points

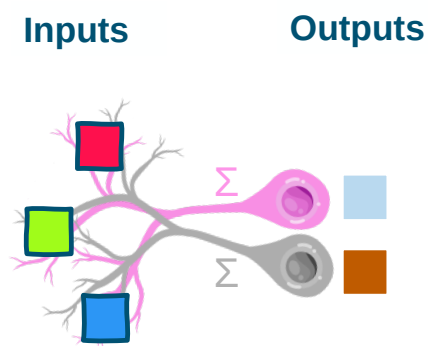


Biological Interpretation of Adding Signals

Matrix vector multiplication



Biological Interpretation

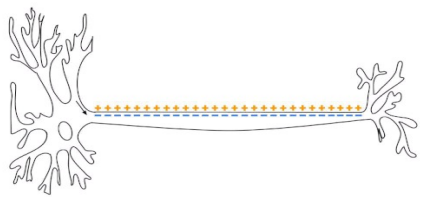


Neuron Image: Wikipedia by Hariadhi - Own work

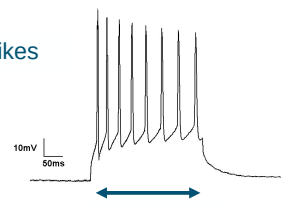
Activation Functions – Biological Interpretation

Image: Wikipedia Laurentaylorj
CC BY-SA 3.0

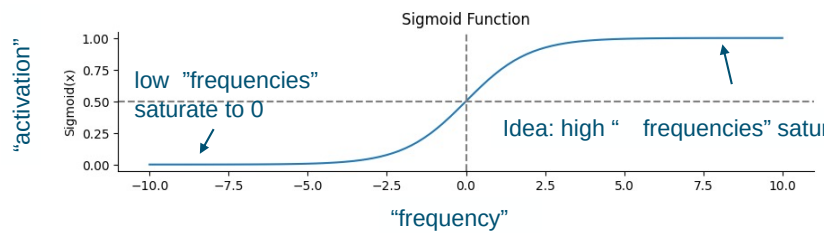
Spikes in voltage
Image: Wikipedia Bilz0r CC BY -SA 3.0



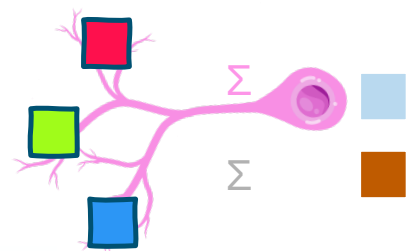
Voltage Spikes



logistic sigmoid $\sigma(x) = \frac{e^x}{1+e^x}$

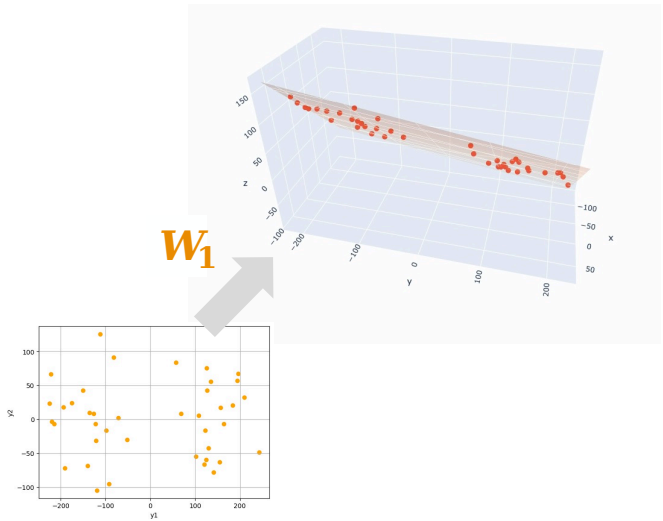


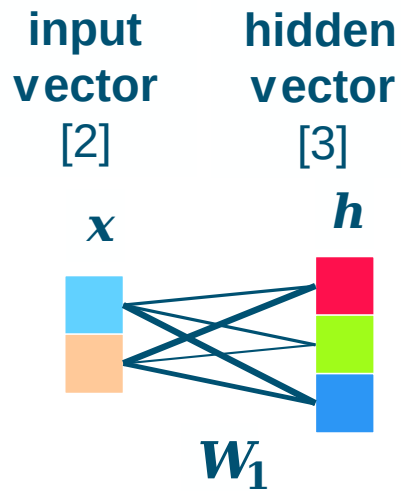
Inputs **Outputs**



Linear Alebra: Stacking Layers

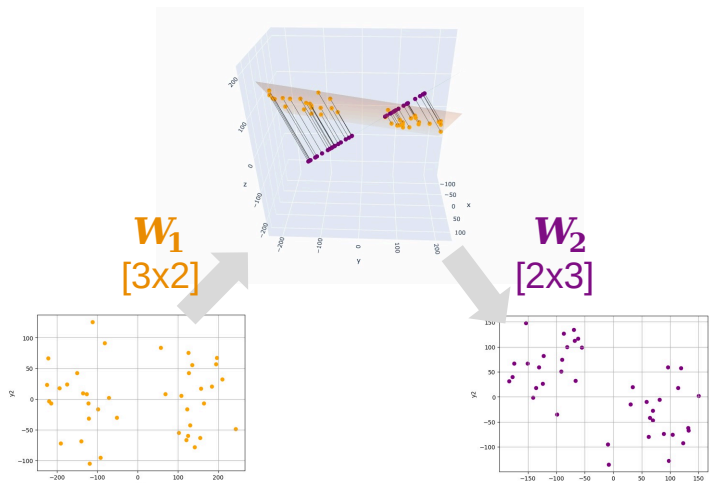
A 2D→3D Linear Projection moves data into 3D representation space

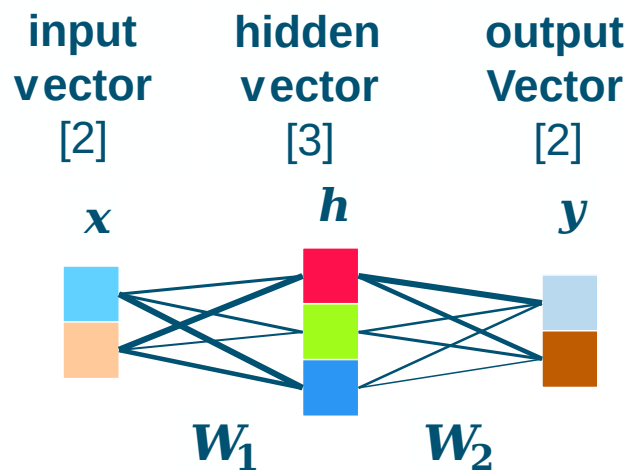




Linear Alebra: Stacking Layers

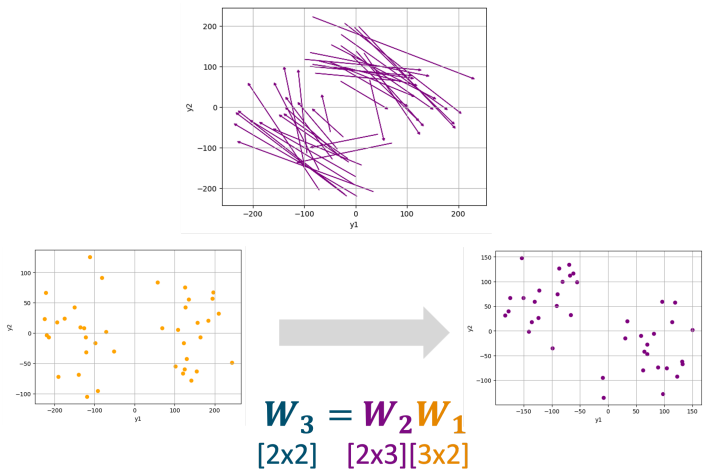
Two Linear projections 2D → 3D → 2D in a 2-Layer network

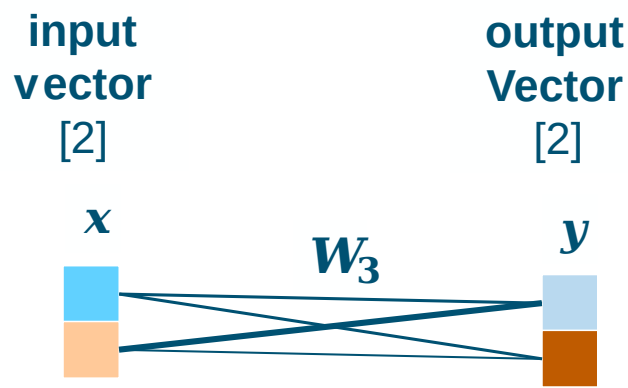




Linear Alebra: Stacking Layers

Two Linear projections 2D → 3D → 2D without non-linearity is a single 2D → 2D projection

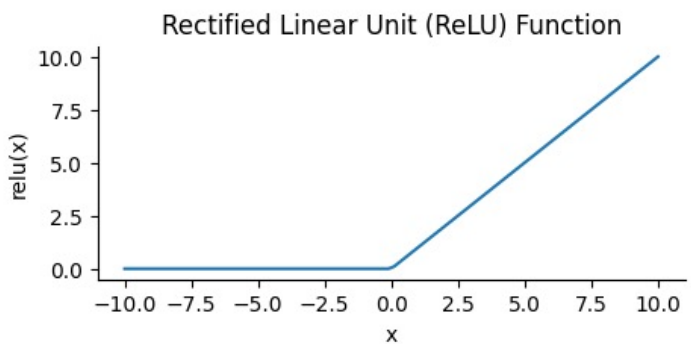


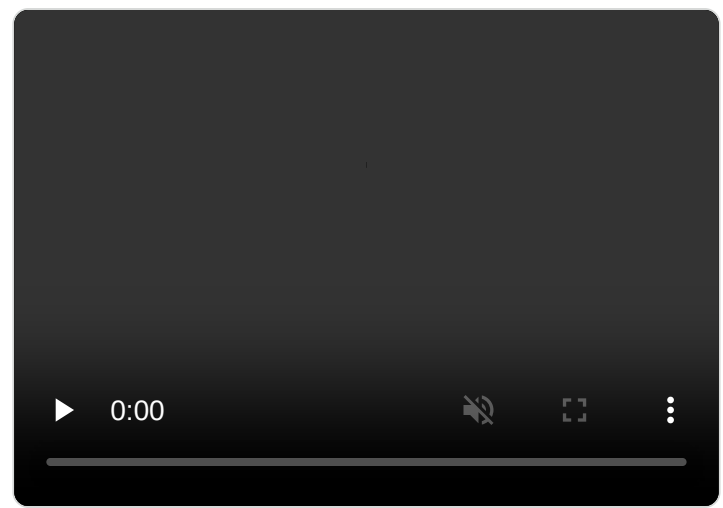


Linear Alebra: Stacking Layers

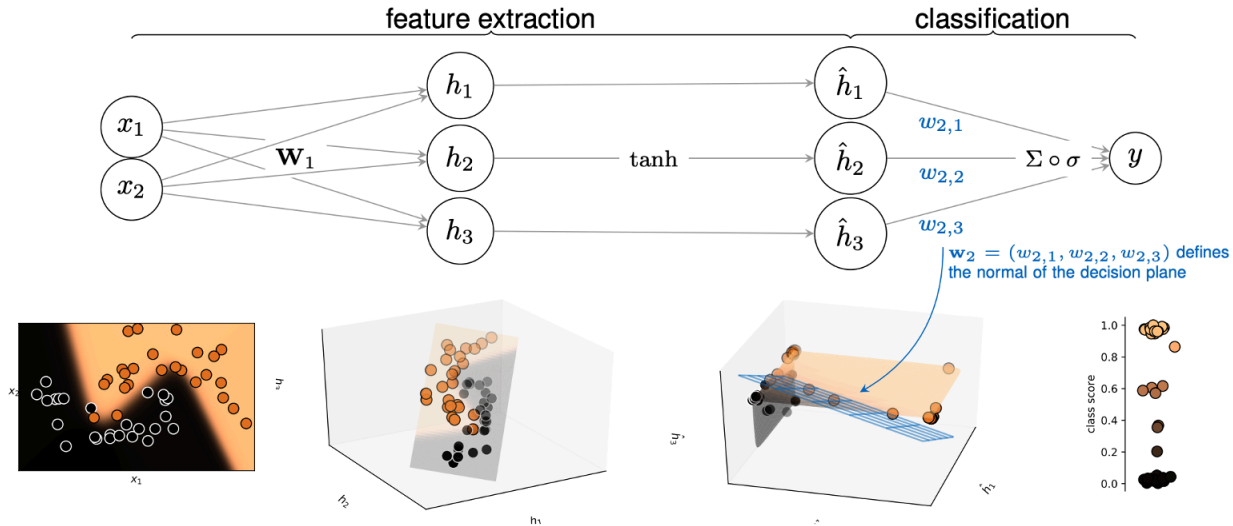
Non-linear functions like ReLU are needed to transform feature space

Rectified Linear Unit: $\mathrm{ReLU}(x) = \max(x, 0)$





More layers → more complex transformations



(a) Input space $\mathbf{x} = (x_1, x_2)$

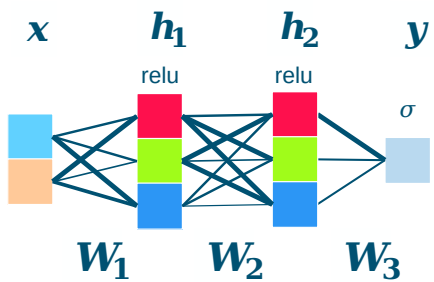
(b) linear projection $\mathbf{w}_1^T \mathbf{x}$ into $\mathbb{R}^3$

(c) non-linear distortion with $\tanh(\mathbf{w}_1^T \mathbf{x})$

(d) Output class probabilities $y \in [0, 1]$

Multi-Layer Perceptron - Implementation

3 Layer MLP Classification Network:




```python
import torch
import torch.nn as nn
import torch.nn.functional as F

class MLPClassifier(nn.Module):
    def __init__(
        self, input_dim, hidden_dim, num_classes
    ):
        super(MLPClassifier, self).__init__()
        self.fc1 = nn.Linear(input_dim, hidden_dim)
        self.fc2 = nn.Linear(hidden_dim, hidden_dim)
        self.fc3 = nn.Linear(hidden_dim, num_classes)

    def forward(self, x):
        x = F.relu(self.fc1(x))      # 1st layer ReLU
        x = F.relu(self.fc2(x))      # 2nd layer ReLU
        x = F.sigmoid(self.fc3(x))   # Output (probs)
        return x
```

Demo: Multi-Layer Perceptron Feature Space

Circle | XOR | Two clusters

Train | Step | Reset model

Rate 0.03

Step 0 · loss 0.716

Input (2)

Dense 1 (6) − + ×

tanh 1 (6) tanh

Dense 2 (2) − + ×

tanh 2 (2) tanh

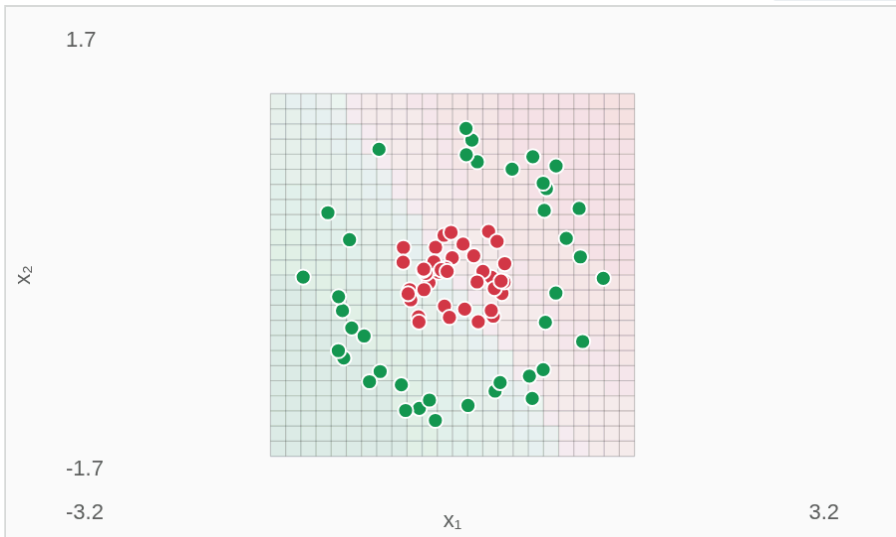
Output (2 logits) → softmax

+ Hidden block

Edits rebuild with fixed seed. Reset keeps data.
Preset buttons restore the original data.

Input — neurons 1 and 2 of 2

Cycle pair



Hover a sample to follow it through the network.
Click: add A · Shift+click: add B · Ctrl/⌘+click: remove nearest
● A ● B · Background: final prediction · Lines: transformed input grid

Inspired by Andrej Karpathy's ConvNetJS Classify2D demo. Implemented with Claude and Codex.

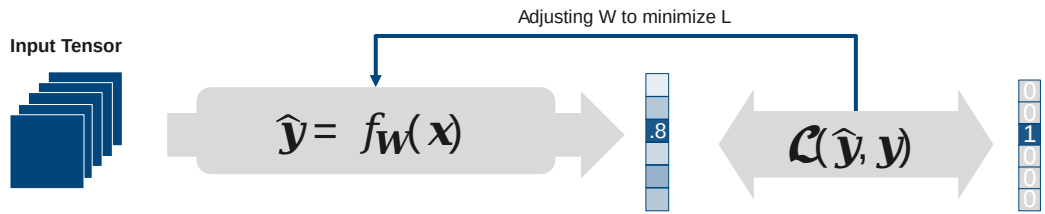
Gradient Descent & Backpropagation

Learning the parameters.

parameters → prediction → loss → gradient → update

Training: Adjusting Weights to Minimize Loss

Finding good parameters W



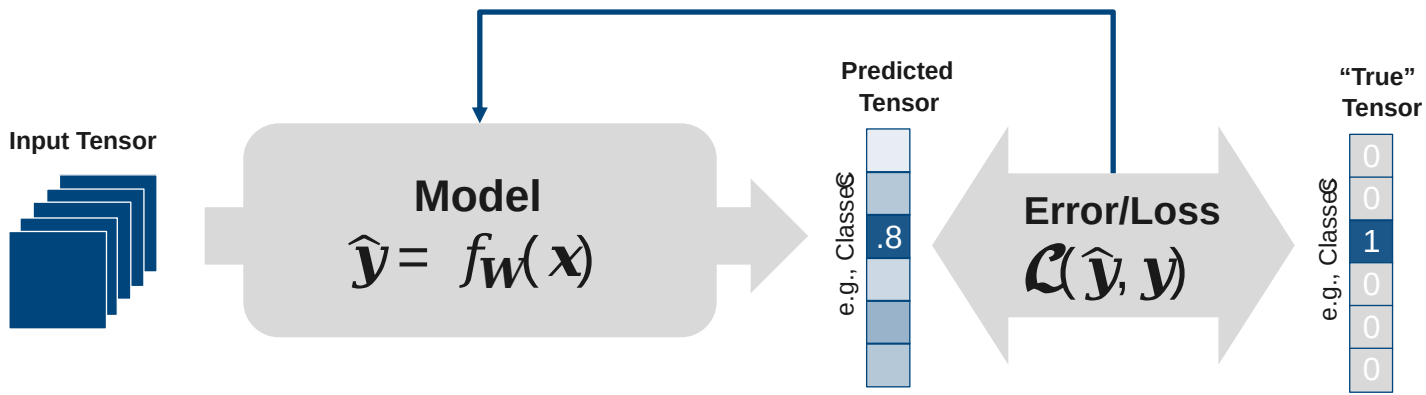
$$\mathcal{L}(\mathcal{D}_{\text{train}}) = \sum_{x,y \in \mathcal{D}_{\text{train}}} \mathcal{L}(f(x), y)$$

Training Dataset $\mathcal{D}_{\text{train}}$

The Learning Objective: argmin

Minimize Error/Loss Function

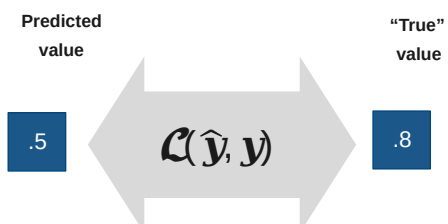
$$W^* = \text{argmin}_W \mathcal{L}(f(x), y)$$

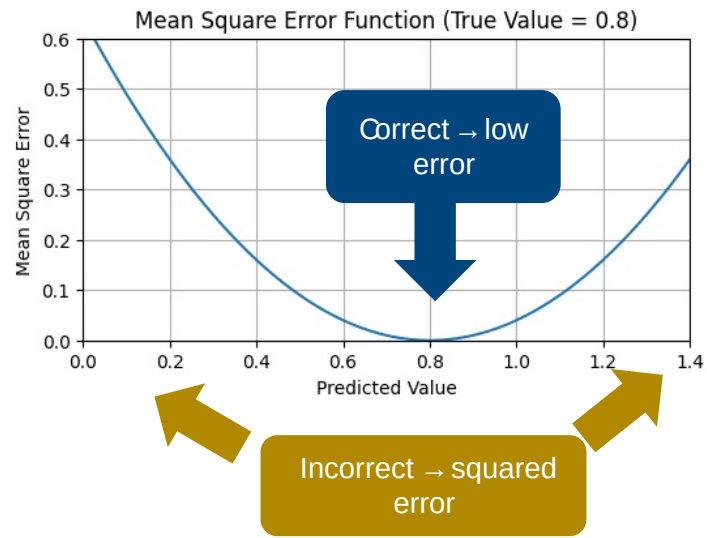


Loss Function: Mean Squared Error

Regression: Mean Squared Error

$$\mathcal{L}(y, \hat{y}) = (y - \hat{y})^2$$

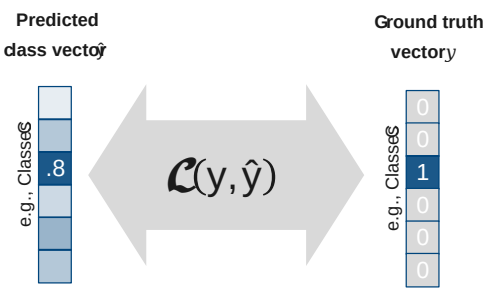


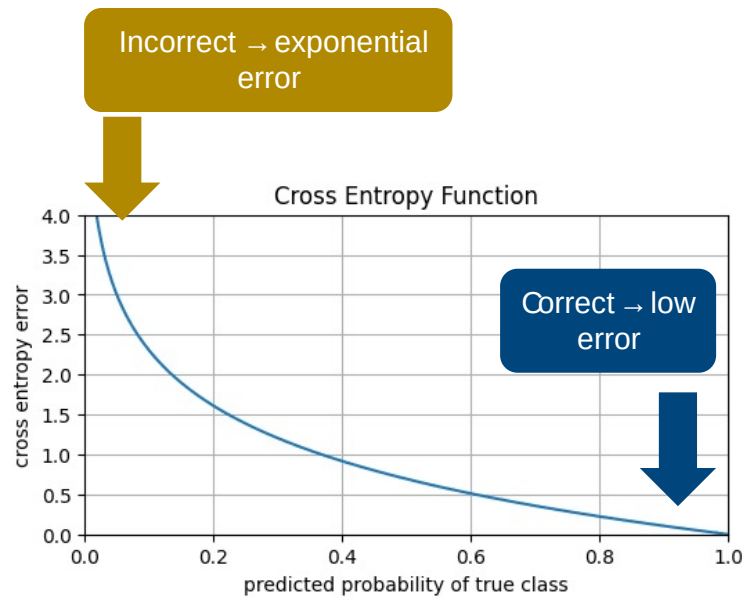


Loss Function: Cross-Entropy

Classification: Cross Entropy

$$\mathcal{L}(y, \hat{y}) = \sum_{c \in \text{Classes}} -y_c \log(\hat{y}_c)$$





Loss Function: Cross-Entropy

Minimize Error/Loss Function:

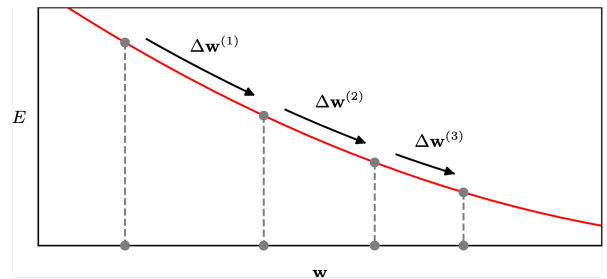
$$W^* = \text{argmin}_W \mathcal{L}(f_W(x), y)$$

Through Iterative Gradient Descent:

$$W^{(i+1)} = W^i - \nabla \mathcal{L}(f_W, \mathcal{D}_{\text{train}})$$

new weights

old weights



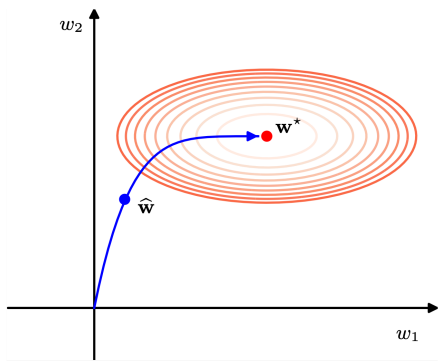
Gradients ∇ of the Loss function $\mathcal{L}$ over a "batch" of training data $\mathcal{D}_{\text{train}}$

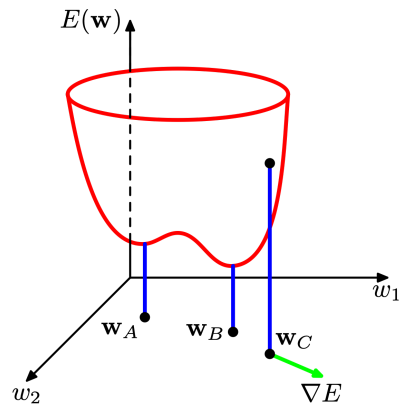
"What weights change the loss the most?"

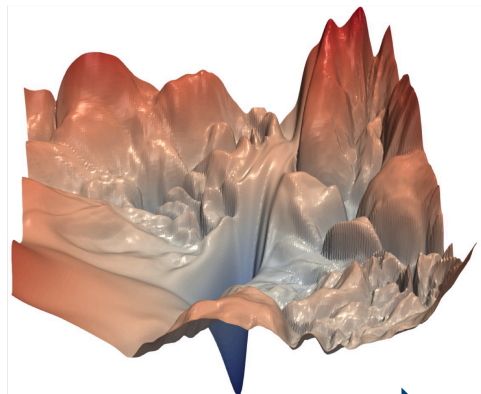
Loss Surfaces

Minimize Error/Loss Function:

$$\boldsymbol{W}^* = \text{argmin}_{\boldsymbol{W}} \boldsymbol{\mathcal{L}}(f_{\boldsymbol{W}}(\boldsymbol{x}), \boldsymbol{y})$$





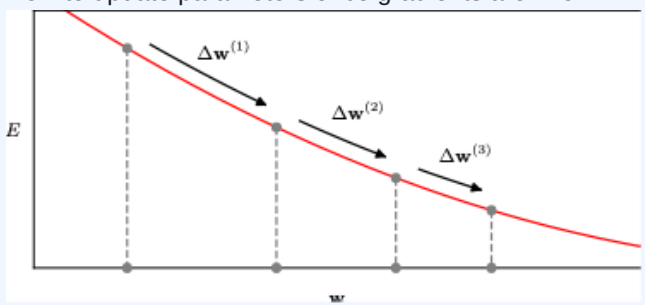


more complex models have more complex loss surfaces

Gradient Descent vs Backpropagation

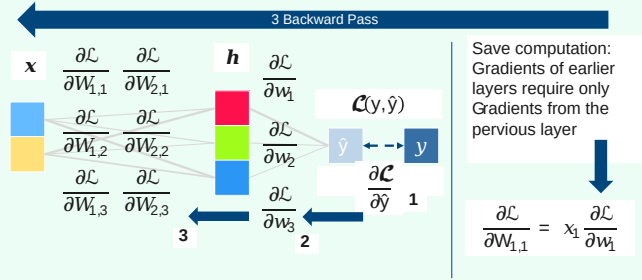
Gradient Descent

How to update parameters once gradients are known.



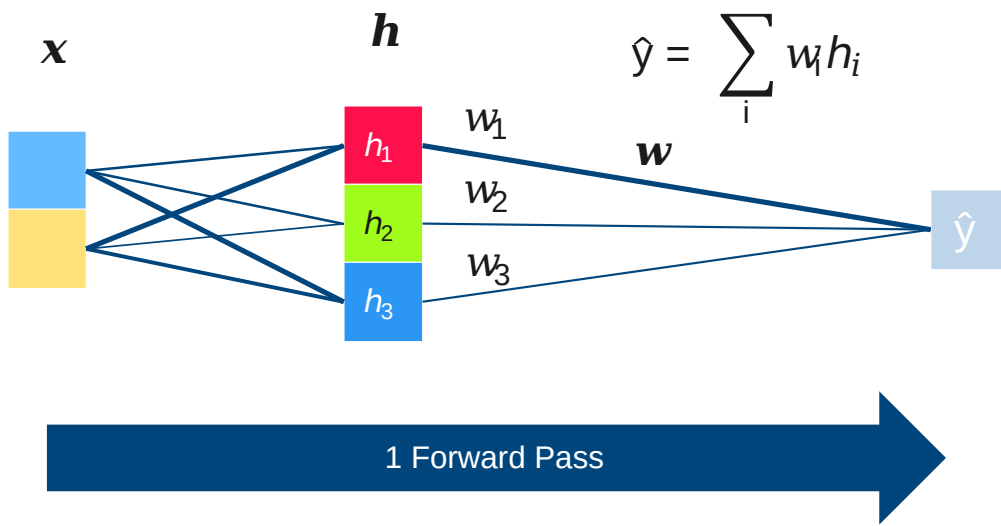
Backpropagation

How to compute gradients efficiently through many layers.

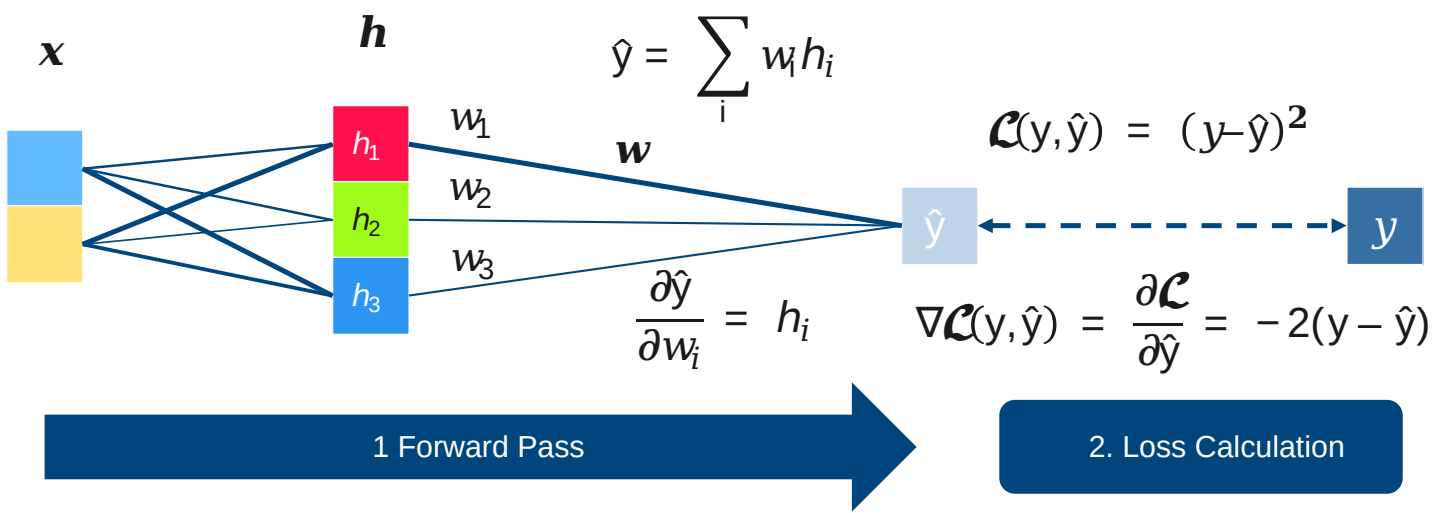


Backprop gives the gradients; gradient descent uses them to step the parameters.

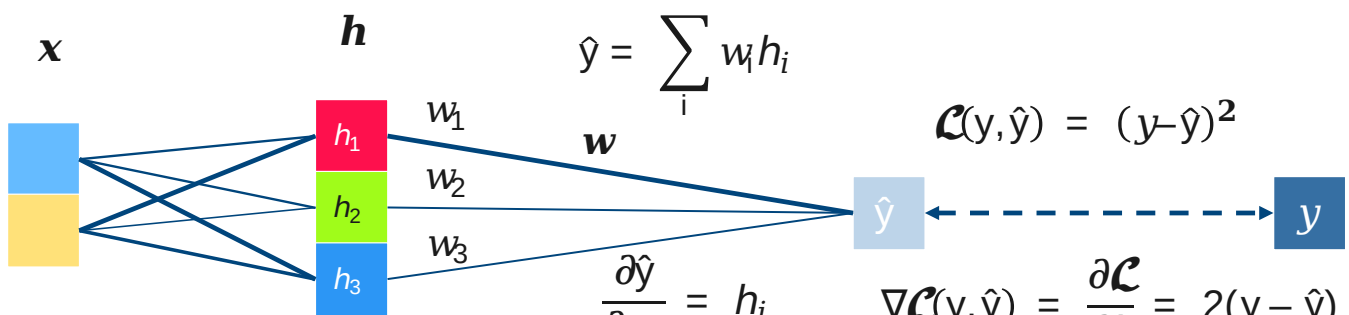
Backpropagation 1



Backpropagation 2



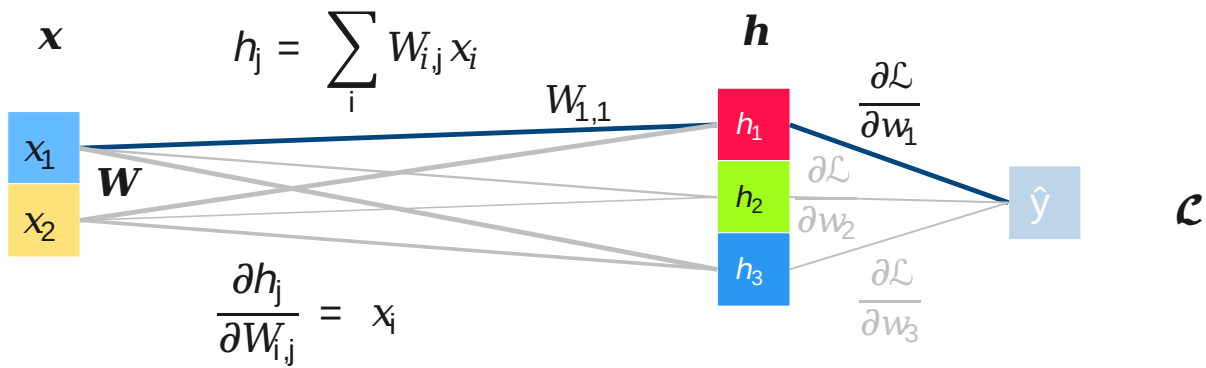
Backpropagation 3



Chain Rule $\frac{\partial z}{\partial x} = \frac{\partial z}{\partial y} \cdot \frac{\partial y}{\partial z}$

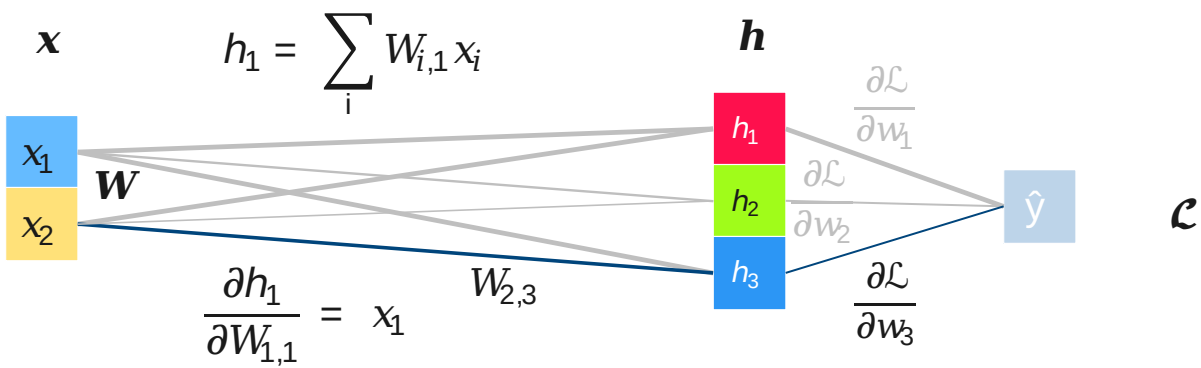
$$\frac{\partial \mathcal{L}}{\partial w_i} = -2(y-\hat{y})h_i$$

Backpropagation 4



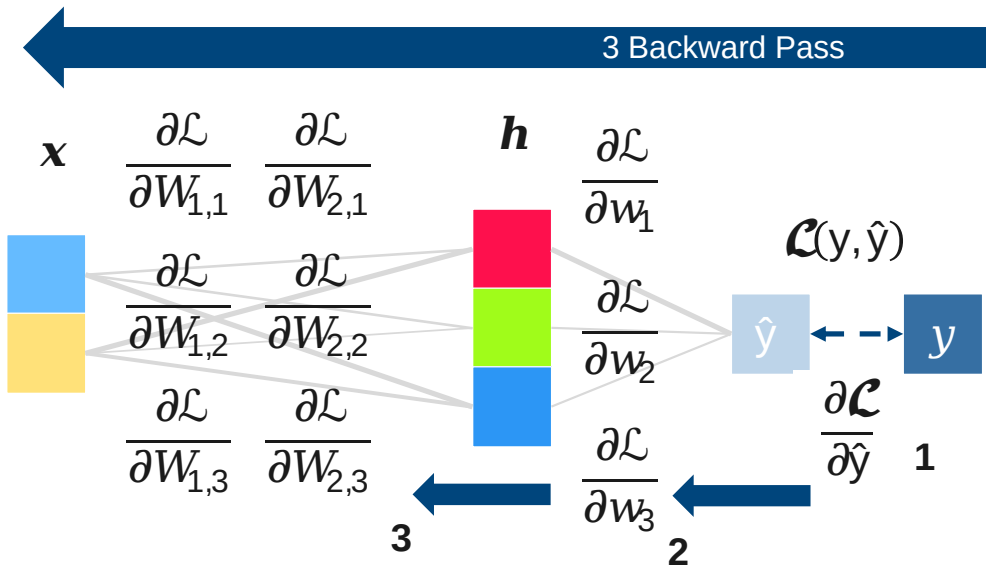
$$\frac{\partial \mathcal{L}}{\partial W_{1,1}} = x_1 \frac{\partial \mathcal{L}}{\partial w_1} = -x_1 2(y - \hat{y}) h_1$$

Backpropagation 5



$$\frac{\partial \mathcal{L}}{\partial W_{2,3}} = x_2 \frac{\partial \mathcal{L}}{\partial w_3} = -x_2 2(y - \hat{y}) h_3$$

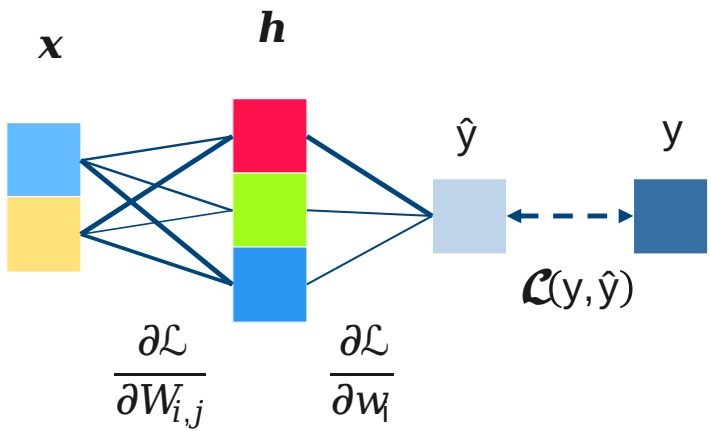
Backpropagation 6



Save computation: Gradients of earlier layers require only Gradients from the pervious layer

$$\frac{\partial \mathcal{L}}{\partial W_{1,1}} = x_1 \frac{\partial \mathcal{L}}{\partial w_1}$$

Backpropagation 7



$$W_{i,j} \leftarrow W_{i,j} - \frac{\partial \mathcal{L}}{\partial W_{i,j}}$$

PyTorch

 # Forward Pass
y_pred = model(x)

 # Loss Computation
loss = MSE(y_pred , y)

 # Backward Pass
loss.backward ()

 # Update Gradients
optimizer.step ()